

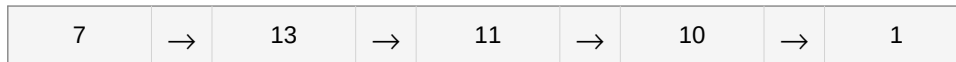
Copy List with Random Pointer

My Personal DSA Notes — Deep Copy of a Linked List with Random Pointers

1. Problem Statement

Each node contains a value, a **next** pointer, and a **random** pointer. The random pointer may point to any node or null. Create a completely separate copy with the same values, next relationships, and random relationships, then return the copied head.

Example next chain



2. Why Normal Copying Is Not Enough

For a normal linked list, copying value and next is enough. Here, a random pointer can point to a node that has not been copied yet. We therefore need a way to find the copy corresponding to every original node.

3. Approach 1 — HashMap

Store **original node** → **copied node** in a HashMap. The video emphasizes storing node references, not just values.

```
HashMap<ListNode, ListNode> map = new HashMap<>();
```

Phase 1: create every copy and store it in the map.

Phase 2: traverse again and connect each copy's next and random pointers using the map.

4. HashMap — Complete Code

```
class Solution {
    public ListNode copyRandomList(ListNode head) {
        if (head == null) return null;

        HashMap<ListNode, ListNode> map = new HashMap<>();

        // Create one copy for every original node.
        ListNode temp = head;
        while (temp != null) {
            ListNode copyNode = new ListNode(temp.val);
            map.put(temp, copyNode);
            temp = temp.next;
        }

        // Connect next and random pointers.
        temp = head;
        while (temp != null) {
            ListNode copyNode = map.get(temp);
            copyNode.next = map.get(temp.next);
            copyNode.random = map.get(temp.random);
            temp = temp.next;
        }

        return map.get(head);
    }
}
```

Assuming average $O(1)$ HashMap operations, two traversals give $O(2n) = O(n)$ time. The HashMap uses $O(n)$ extra space. The copied nodes themselves also require $O(n)$ memory because the problem requires a new list.

5. Why Optimize?

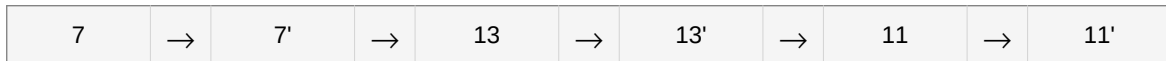
The HashMap is extra memory. The video removes that HashMap by placing each copied node directly between its original node and its original next node.

6. Optimized Idea — Interleave the Copies

Original



After inserting copies



Now the copy of an original node is always **original.next**. This gives us the mapping without a HashMap.

7. Phase 1 — Insert Copies

```
ListNode temp = head;

while (temp != null) {
    ListNode copyNode = new ListNode(temp.val);

    copyNode.next = temp.next;
    temp.next = copyNode;

    // Skip the copy and reach the next original node.
    temp = temp.next.next;
}
```

The two-step movement is important: after insertion, temp.next is the copied node, so temp.next.next reaches the next original node.

8. Phase 2 — Connect Random Pointers

```
temp = head;

while (temp != null) {
    ListNode copyNode = temp.next;

    if (temp.random != null) {
        // The copy of temp.random is immediately after it.
        copyNode.random = temp.random.next;
    }

    temp = temp.next.next;
}
```

If original A.random points to original B, then B.next is the copy of B. Therefore A's copy can set its random pointer to A.random.next. The null check is required because random may be null.

9. Phase 3 — Separate the Lists

```
ListNode dummy = new ListNode(-1);
ListNode res = dummy;
temp = head;

while (temp != null) {
    // Add the copied node to the result list.
    res.next = temp.next;

    // Restore the original list's next pointer.
    temp.next = temp.next.next;

    res = res.next;
    temp = temp.next;
}
```

```
return dummy.next;
```

After this phase, the original list is restored and the copied nodes form a separate list. `dummy.next` is the copied head.

10. Complete Optimized Java Solution

```
class Solution {
    public ListNode copyRandomList(ListNode head) {

        if (head == null) {
            return null;
        }

        // Phase 1: insert a copy after every original node.
        ListNode temp = head;
        while (temp != null) {
            ListNode copyNode = new ListNode(temp.val);

            copyNode.next = temp.next;
            temp.next = copyNode;

            // Move to the next original node.
            temp = temp.next.next;
        }

        // Phase 2: connect random pointers.
        temp = head;
        while (temp != null) {
            ListNode copyNode = temp.next;

            if (temp.random != null) {
                // temp.random.next is the copy of the random target.
                copyNode.random = temp.random.next;
            }

            temp = temp.next.next;
        }

        // Phase 3: extract the copied list and restore the original.
        ListNode dummy = new ListNode(-1);
        ListNode res = dummy;
        temp = head;

        while (temp != null) {
            res.next = temp.next;
            temp.next = temp.next.next;

            res = res.next;
            temp = temp.next;
        }

        return dummy.next;
    }
}
```

11. Full Dry Run

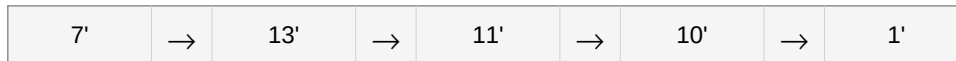
The process has three clear phases.

After Phase 1

7'	→	13	→	13'	→	11	→	11'	→	10	→	10'	→	1
----	---	----	---	-----	---	----	---	-----	---	----	---	-----	---	---

Phase 2: connect every copied random pointer. For example, if 13.random points to 7, then the copy of 13 points to 7.next, which is the copy of 7.

Copied list after Phase 3



12. Why temp.next.next?

After interleaving, the structure is original → copy → original → copy. Therefore:

```
temp = temp.next.next;
```

moves from one original node to the next original node. Moving only temp.next would move onto the copy.

13. Why temp.random.next?

Suppose A.random points to B. Because every copy is placed after its original:

Original target and its copy

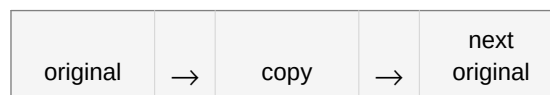


So the copy of B is B.next. Therefore:

```
copyA.random = A.random.next;
```

14. Why Separate and Restore?

Interleaved



The extraction step does two things at once:

- res.next = temp.next attaches the copy to the new copied list.
- temp.next = temp.next.next restores the original node's next pointer.

Once all nodes are processed, the original list is back to its original next-chain and the copied list is independent.

15. Optimized Complexity — Clearly

The video divides the optimized solution into three traversals:

Phase 1: insert copies → $O(n)$

Phase 2: connect random pointers → $O(n)$

Phase 3: separate/restore → $O(n)$

Total = $O(3n)$ = **$O(n)$** time.

For space, there is no HashMap. The problem itself requires n new copied nodes, so the copied list takes **$O(n)$** memory. The video reports $O(n)$ space for this required new list and treats it separately from avoidable auxiliary space.

16. HashMap vs Optimized

Approach	Time	Auxiliary Structure	Main Idea
HashMap	$O(n)$	$O(n)$ HashMap	Original node → copy
Interleaving	$O(n)$	No HashMap	Copy placed after original

The optimized method removes the HashMap, while the $O(n)$ copied nodes cannot be avoided because the task explicitly asks for a deep copy.

17. Edge Cases

- Empty list → return null.
- One node → copy it; random may point to itself or null.
- random == null → copied random remains null.
- random points to itself → copied random points to the copied node itself.
- Random points forward/backward → interleaving handles both.
- Arbitrary random relationships → no restriction on where random points.

18. Important Lines to Remember

```
// Insert copy
copyNode.next = temp.next;
temp.next = copyNode;

// Move to next original
temp = temp.next.next;

// Connect random
copyNode.random = temp.random.next;

// Extract copy
res.next = temp.next;

// Restore original
temp.next = temp.next.next;

// Copied head
return dummy.next;
```

19. Final Intuition

The key trick is: **put every copy immediately after its original**. Then the copy of any random target is always one step away. After connecting random pointers, separate the interleaved structure into two independent lists and restore the original list.

20. Quick Revision

HashMap: create copies → store original-to-copy mapping → connect next/random → return map.get(head).
Time $O(n)$, HashMap space $O(n)$.

Optimized: insert copies → connect random using target.next → separate and restore. Time $O(n)$, no HashMap, $O(n)$ memory for the required copied list.

Core trick: original.next is its copy while the lists are interleaved.